

■ 2.1.15 Advanced Topic: Applying Functions with Side Effects

<code>Scan[f, expr]</code>	evaluates f applied to each element of $expr$ in turn
<code>Scan[f, expr, lev]</code>	evaluates f applied to parts of $expr$ on levels specified by lev

Evaluating functions on parts of expressions.

Here is a list of three elements.

```
In[1]:= t = {a, b, c}
Out[1]= {a, b, c}
```

`Map` constructs a new list in which `f` has been applied to each element of `t`.

```
In[2]:= Map[f, t]
Out[2]= {f[a], f[b], f[c]}
```

`Scan` evaluates the result of applying a function to each element, but does not construct a new expression.

```
In[3]:= Scan[Print, t]
a
b
c
```

Functions like `Map` allow you to create expressions with parts modified. Sometimes you simply want to go through an expression, applying a particular function to some parts of it, but not building a new expression. A typical case is when the function you apply has certain “side effects”, such as making assignments, or generating output.

`Scan` visits the parts of an expression in a depth-first walk, with the leaves visited first.

```
In[4]:= Scan[Print, 1 + x^2, Infinity]
1
x
2
x
```

You can use `Return` to stop scanning the expression tree.

```
In[5]:= Scan[(Print[#]; If[# > 2, Return[found]]) &,
              {1, 2, 3, 4}]
1
2
3
Out[5]= found
```